DevOps Tool Comparison Dashboard

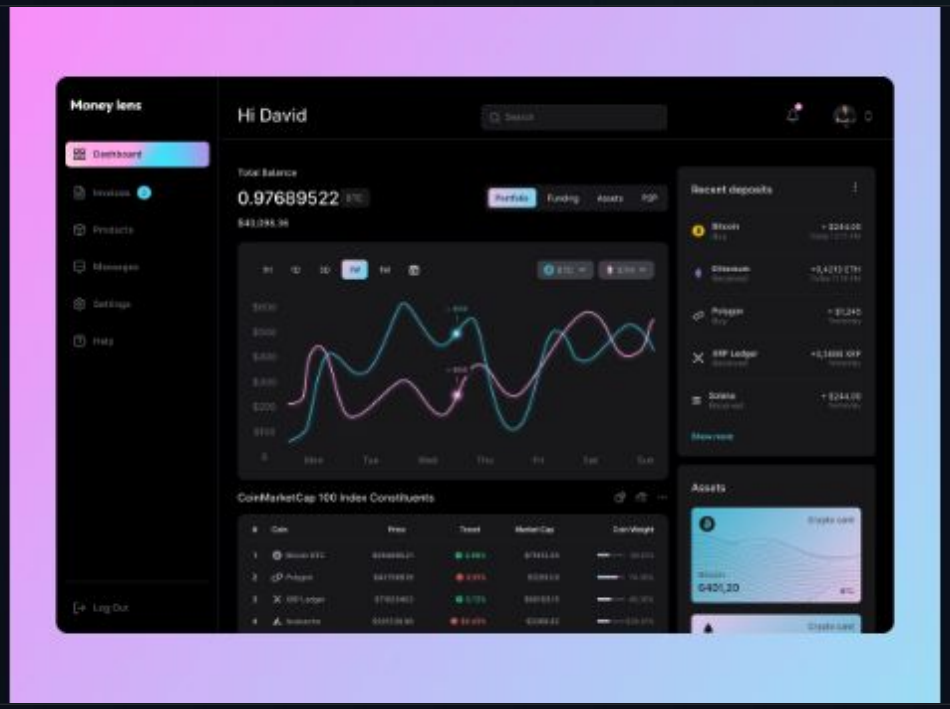
A Complete End-to-End DevOps Lifecycle Implementation

Project Overview

Bridging the gap between software development and IT operations through an automated, monitored, and containerized web application.

The Application

- **Dynamic Catalog:** A full-stack web application designed to help users browse, search, and filter DevOps tools.
- **Responsive UI:** Built with Vanilla JavaScript and Bootstrap 5 for real-time interaction without page reloads.
- **Robust Backend:** Powered by Node.js and Express.js, providing a scalable foundation for RESTful APIs.
- **Data Enrichment:** Seamlessly combines local file storage with live third-party API data.



Core Technologies

Backend

Node.js and Express.js handle API routing, CSV data parsing, and asynchronous requests to external services.

Containerization

Docker encapsulates the environment using an optimized Alpine Linux base image to ensure consistency.

Automation

Jenkins orchestrates the CI/CD pipeline, and Puppet manages infrastructure configuration.

Sprints 1 & 2: Core Logic & API

Static Data

The application parses a local devops_data.csv file into memory on startup using the csv-parser module, providing instant access to tool features and pricing.

Live Integration

Using axios, the backend reaches out to the GitHub REST API to fetch live stargazers_count metrics, merging them dynamically with the static dataset before sending to the client.

Sprint 3: Containerization & CM

We solved the "it works on my machine" problem by containerizing the application using a highly optimized Dockerfile based on node:14-alpine.

Additionally, we embraced Infrastructure as Code (IaC) by writing a declarative Puppet manifest (install_docker.pp) to automate the deployment of the Docker engine on any target server.

Sprint 4: CI/CD Pipeline

- **1. Checkout SCM:** The Jenkins Declarative Pipeline automatically pulls the latest commit from the main branch on GitHub.
- **2. Install & Test:** Jenkins provisions the environment, installs Node dependencies, and executes npm test to verify data integrity.
- **3. Build Image:** Upon successful testing, Jenkins builds the Docker container and tags it uniquely using the environment BUILD_ID.
- **4. Push to Registry:** Using secured credential bindings, the pipeline logs into Docker Hub and pushes the immutable image artifact.

Pipeline Efficiency

4

Automated Stages

0

Manual Interventions Needed

From a simple code commit to a fully tested, containerized artifact residing in a remote registry—all handled autonomously.

Sprint 5: Cloud Deployment

Serverless Container Hosting

The application is deployed directly to the cloud using Render's Web Service platform. Instead of managing raw EC2 instances, Render is configured to automatically pull our latest tagged image straight from Docker Hub.

This approach perfectly bridges our Jenkins CI pipeline with continuous cloud delivery, ensuring zero downtime and simplified infrastructure management.

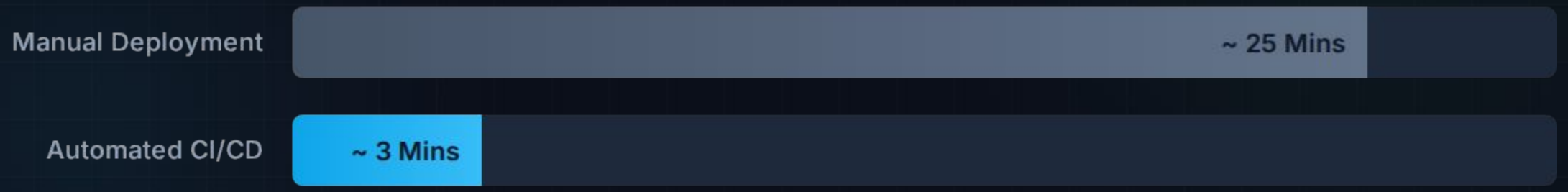
Sprint 5: System Monitoring

Metrics Endpoint: Configured prom-client to expose internal system metrics.

Prometheus Scraping: Local instance configured to ping the cloud URL every 15s.

Health Tracking: Monitoring RAM usage, CPU time, and Node event loop lag.

Workflow Speed Comparison



By implementing Jenkins, Docker, and Render, we reduced the release cycle duration by over 85%, eliminating human error and ensuring consistent artifact builds.